Critical Evaluation of “A Subset Feature Elimination Mechanism for Intrusion Detection System”

Final Project — Data Science in Cyber (Dr. Uri Itai)

Adam Karain (ID 318407889) — June 2026

Evaluated source: H. Nkiama, S. Z. Mohd Said and M. Saidu, “A Subset Feature Elimination Mechanism for Intrusion Detection System,” *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 7, no. 4, 2016. The original authors released no code; the GitHub repository *CynthiaKoopman/Network-Intrusion-Detection* is an independent third-party reproduction used here only as corroborating reference. Dataset: NSL-KDD (mirror: *defcom17/NSL_KDD*). All code, data files and the executed notebook accompanying this report are available in the submission repository.

1. Summary of the Source

The problem being addressed. A network Intrusion Detection System (IDS) must decide, for every connection it observes, whether the traffic is benign or malicious. Modern networks generate traffic at a volume that makes both the speed and the accuracy of this decision critical: a slow detector cannot keep up, and a detector that misses attacks creates false confidence. The article addresses a specific facet of this problem — dimensionality. The NSL-KDD benchmark describes every connection with 41 features, and the authors argue that many of them are redundant or irrelevant, slowing down the classifier and potentially hurting its detection rate.

Why the problem is important. Feature selection in an IDS is not a cosmetic optimization. Fewer features mean cheaper feature extraction at wire speed, faster model retraining as traffic evolves, and — if done correctly — models that are easier for security analysts to interpret. The question of *which* features matter is itself of operational value, since it tells analysts what to monitor.

The proposed solution. The authors propose a two-stage feature-selection mechanism, applied separately to each of the four NSL-KDD attack categories (DoS, Probe, R2L, U2R): first a univariate ANOVA F-test filter (scikit-learn’s *SelectPercentile*), then Recursive Feature Elimination (RFE) wrapped around a decision-tree classifier. The pipeline selects 11–15 features per category out of the 121 columns that remain after one-hot encoding. The headline claims are that the reduced models reach approximately 99.7–99.9% accuracy, precision, recall and F-measure, and that training time drops by roughly an order of magnitude (e.g., 15.5 s to 0.96 s for the DoS class).

The dataset used. NSL-KDD (Tavallaee et al., 2009), the cleaned successor of KDD’99: 125,973 labelled training connections (*KDDTrain+*) and 22,544 test connections (*KDDTest+*), each with 41 features, an attack-type label (39 types grouped into DoS, Probe, R2L, U2R and normal) and a difficulty score. By deliberate design of its authors, *KDDTest+* contains 17 attack types that never appear in training — 29.2% of all attack records — so that the benchmark measures generalization to novel attacks, not only recognition of known ones.

The model and methodology employed. A CART decision tree (scikit-learn *DecisionTreeClassifier*, information-gain criterion) is trained *per attack category as a binary problem* (normal vs. that class) on one-hot-encoded, standardized features, with the two-stage

feature selection described above. On evaluation the article is explicit (Methodology, Step 5): “the test data was used to make prediction of our model and for evaluation,” and “a 10-fold cross-validation was performed during all the process”; results are reported as accuracy, precision, recall, F-measure and confusion matrices. As we show in Sections 2 and 5, the per-class binary framing and the pooled cross-validation — not an untouched test set — are what make the reported numbers so high.

2. Critical Evaluation

The main claims. We extract three explicit claims and add one question the article leaves open. C1: the two-stage feature selection improves classification performance (the article’s Tables IV–V show R2L accuracy rising from 97.03% with 41 features to 99.88% with 13). C2: the per-class detectors achieve ≈99.7–99.9% accuracy, precision and recall. C3: feature selection drastically reduces training time (Table IX). Q4 (our extension, not a claim the article makes): does the per-class evaluation reflect held-out detection on the benchmark’s official test set?

Are the claims supported by the presented evidence? Internally, yes — and we reproduce them: cross-validating within a single distribution returns 99.6–99.9% per class, matching the article (for U2R, 99.9% vs. a claimed 99.95%). The numbers are real, not fabricated. The decisive question is the evaluation *protocol*. The article evaluates each attack class as a binary problem under 10-fold cross-validation, which pools all of a class’s records into the same fitting-and-scoring loop; that protocol answers “can the model re-recognize attacks drawn from its own distribution?”. The article’s conclusions, however, speak about detecting intrusions in general — a claim this evaluation does not test.

Is the evaluation methodology appropriate? No, in three respects. First — and this corrects an error in the previous version of this project — the issue is not that the test set is untouched. The article’s text says the test data was used, and its confusion-matrix row totals (7,460 DoS; 2,421 Probe; 2,885 R2L; 67 U2R) equal the official *KDDTest+* class sizes exactly. The flaw is that the test set is *cross-validated rather than held out*: a 10-fold CV that includes the test records pools the very novel attacks *KDDTest+* was built to isolate into the model’s own training folds, so the train→test novelty gap is never exercised. Second, the article relies on accuracy as its headline metric despite severe class imbalance (52 U2R records in 125,973, i.e. 0.04%): a trivial all-negative classifier reaches 99.96% accuracy on the U2R task, so third-decimal accuracy differences carry no information about detection ability. Third, the bookkeeping is inconsistent: the article reports 126,620 training and 22,850 test records, while the official files contain 125,973 and 22,544; its Table III lists 3,191 R2L test instances, yet its own R2L confusion matrix sums to 2,885 (the value we obtain from the official file).

What mechanism produces the published numbers? We can demonstrate it directly. Running 10-fold cross-validation *on KDDTest+ itself* — the procedure visible in the third-party reference implementation, which calls *cross_val_score* on the test set — reproduces the article almost exactly: our R2L figure under that protocol is **97.7%** against the article’s claimed 97.4%. We are careful with attribution: the original authors released no code, so we cannot prove they ran this exact procedure, only that it is the procedure consistent with both their test-set-sized confusion matrices and their cross-validation-level metrics. Held out honestly (fit on *KDDTrain+*, predict *KDDTest+* once), the same faithful pipeline detects only 11.1% of R2L and 34.3% of U2R attacks.

Possible weaknesses and limitations. Beyond the evaluation protocol: class imbalance is never discussed or mitigated (no class weights, no resampling, no cost-sensitive analysis); the train/test

prevalence shift (R2L is 0.8% of training but 12.8% of the test set) goes unmentioned; no variance or significance is attached to the third-decimal accuracy improvements credited to feature selection; and the released materials are incomplete (Section 4 of this report).

Are the conclusions justified? The speed conclusion (C3) is justified and we confirm it. So is C1 *within the distribution*: with the ANOVA + RFE selector, 11–15 of 121 columns preserve performance, and the article's premise that NSL-KDD is highly redundant is correct. What the evidence does not support is the leap from those in-distribution numbers (C2) to detecting intrusions in general (the spirit of Q4): under the benchmark's intended held-out protocol the identical faithful pipeline reaches only 11.1% R2L recall (95% CI ≈ 10–12%) against a claimed 97.4%. The disagreement is methodological, not a failure to reproduce — we reproduce the article's numbers first, then show they do not survive the held-out split the benchmark was built around.

3. Feature Engineering Analysis

Was feature engineering performed, and which features were used? Yes, on two levels. The dataset itself is already heavily engineered: NSL-KDD's 41 features include basic TCP/IP fields (duration, protocol, service, flag, byte counts), content features extracted from payloads (failed logins, root shells, file creations), and statistical window features computed over the preceding 2 seconds (*count*, *serror_rate*, …) or the last 100 connections per host (*dst_host_**). The article adds three transformations: one-hot encoding of the three nominal features (*protocol_type*, *service*, *flag*; 41 → 121 columns), standardization to zero mean and unit variance, and the two-stage feature selection (ANOVA filter + RFE) that is its main contribution.

Transformations in our reproduction, and why. To stay faithful to the article our design matrix uses exactly its two preprocessing steps: one-hot encoding of the three nominal features (*protocol_type*, *service*, *flag*; 41 → 121 columns) and standardization. One-hot is the right choice — label encoding would fabricate an order and target encoding would inject label information. Standardization is fitted only on the rows a model trains on, inside each pipeline; the third-party reference implementation instead fits a separate scaler on the test set, a real leak we do not reproduce. We also *tested* a *log1p* compression of the four heavy-tailed counters — an obvious candidate, since they span nine orders of magnitude with skewness up to 290 — but adopted it nowhere: on the held-out set it moves logistic-regression recall by under one point and actually *lowers* ROC-AUC (0.780 → 0.764), and the tree models are invariant to any monotonic transform by construction. Reporting *log1p* as an improvement without measuring it would be exactly the kind of unsupported claim this project is meant to catch, so we keep the faithful matrix and record the negative result.

Redundancy in the system. The Spearman correlation screen in the notebook finds near-duplicate groups: the SYN-error rate appears at three aggregation levels (*serror_rate*, *srv_serror_rate*, *dst_host_serror_rate*; pairwise $\rho = 0.92$–0.97), *same_srv_rate* and *diff_srv_rate* are near mirror images ($\rho = -0.92$), and *dst_host_srv_count* tracks *dst_host_same_srv_rate* ($\rho = 0.92$). One would spot such redundancy exactly this way — a $|\rho| > 0.9$ screen, variance-inflation factors, or clustering features on $1-|\rho|$ — and tackle it by keeping one representative per cluster, by regularization, or by letting a wrapper method such as RFE prune the copies. To be fair to the article, this redundancy means its premise (41 features contain substantial redundancy) is correct; our criticism concerns what was measured, not the premise.

Was the feature-selection process meaningful? Mathematically, the ANOVA filter scores each feature by between-class F-statistic and RFE then prunes by impurity contribution — a coherent way to find a small high-signal subset. On the DoS class our independent, faithful run recovers 5 of the article's 12 features, with several of the rest close cousins (it keeps *flag_SF* where the article keeps *flag_S0* — complementary encodings of whether a connection completed or hung half-open, the SYN-flood signature). The shared features are exactly the signatures an analyst would name: the SYN-error window rates (*dst_host_serror_rate*, *dst_host_srv_serror_rate*), *service_ecr_i* (ICMP echo replies — *smurf*), *wrong_fragment* (*teardrop*) and *same_srv_rate*. The selection is interpretable and domain-plausible — and therein lies the caveat: these are signatures of the *specific attacks in the training catalog*. A feature set optimized to separate *neptune* from normal traffic has no reason to flag an *apache2* request flood, which is precisely the failure our error analysis documents.

Features that could improve performance. Payload-derived statistics (byte entropy, header anomalies), per-source aggregates over horizons longer than 2 seconds (failed logins per hour, fan-out), and cross-connection sequence features. None exist in NSL-KDD — an argument for validating on modern datasets (CICIDS2017/2018, UNSW-NB15).

4. Reproducibility Analysis

Can the code be executed successfully? The article releases no code; reproduction depends on its prose plus the community reference implementation. The reference notebook (*DecisionTree_IDS.ipynb*) runs after minor version updates (it predates current scikit-learn APIs). Our own notebook executes end-to-end with fixed seeds ("Run All", approximately two minutes) and reproduces every number quoted in this report.

A note on attribution. The repository we reference, *CynthiaKoopman/Network-Intrusion-Detection*, is an *independent third-party* reproduction, not the authors' own code. We use it only as corroborating evidence for how the article's numbers can arise, and we do not attribute its specific choices to the original authors as established fact. The NSL-KDD files themselves are freely available; we vendor the three the notebook needs in *data/*. The article specifies neither library versions nor random seeds, so exact reproduction of its third-decimal figures is not possible from its text alone.

Do hidden preprocessing steps exist? Yes. Several decisions material to the results are unstated in the article: how one-hot encoding handles the six service categories that occur only in training; whether scaling is fitted per split (the reference code fits a separate scaler on the test set — a leak); and, most consequentially, exactly what the reported metrics are computed on. What the article does state is that the test data was used with 10-fold cross-validation throughout, and its confusion-matrix totals equal the *KDDTest+* class sizes; the only procedure consistent with both a test-set-sized confusion matrix and cross-validation-level metrics is cross-validation applied to the test set — which we demonstrate reproduces the article's R2L figure (97.7% vs. 97.4%), and which the third-party reference performs explicitly.

Overall reproducibility. Mixed. The pipeline is built from standard, well-documented components and we reproduce the published numbers to the second decimal — in that narrow sense reproducibility is excellent. But the evaluation protocol, the single most important methodological fact, is stated only loosely in the article and its record counts disagree with the official files. Our own submission removes that ambiguity: pinned dependencies, vendored data, fixed seeds, leakage-free pipelines, and one clearly labelled protocol per number.

5. Experimental Results

Experiments performed. (i) A binary detector (normal vs. attack) trained on all 121 encoded features with three models — logistic regression, the article's decision tree, and a random forest — each evaluated under two protocols: Protocol A, a stratified 80/20 split inside *KDDTrain+* (statistically equivalent to the article's cross-validation), and Protocol B, the official held-out split (train on *KDDTrain+*, evaluate on *KDDTest+*). (ii) A faithful re-run of the article's per-class pipeline — ANOVA filter *and* RFE around a decision tree, using the article's own per-class feature counts (DoS 12, Probe 15, R2L 13, U2R 11) — scored three ways: 10-fold CV on training data, 10-fold CV on the test data (the reference implementation's procedure), and the held-out split, the last with a bootstrap 95% recall interval. (iii) An error analysis of the strongest model. **Engineering notes:** every model is a leakage-free *Pipeline* (scaler and selector fitted only on training rows); the cross-validation that stands in for the article's protocol runs on a stratified 12,000-record subsample with RFE step 10 for compute reasons (feature ranks are stable). We deliberately add no transform the article lacks.

Evaluation metrics and their justification. Precision (share of alerts that are real attacks — its complement drives analyst alert fatigue); recall, the IDS detection rate (every false negative is an undetected intrusion — in security the costlier error); F1 and F2 (F2 weights recall twice as heavily, encoding the asymmetric cost of a breach versus minutes of triage); the Matthews correlation coefficient, the most reliable single number under class imbalance because it uses all four confusion-matrix cells and cannot be inflated by majority-class guessing; and ROC-AUC, which is threshold-free and separates ranking quality from threshold placement. Accuracy is reported only because the article uses it as its headline figure. Regression metrics do not apply; specificity is captured by the false-positive analysis; F0.5 encodes the opposite cost structure to the IDS use case and is therefore omitted.

Model	Protocol	Accuracy	Precision	Recall	F1	F2	MCC	ROC-AUC
Logistic Regression	A	0.9723	0.9766	0.9636	0.9700	0.9662	0.9444	0.9963
Decision Tree	A	0.9985	0.9980	0.9987	0.9984	0.9986	0.9970	0.9985
Random Forest	A	0.9990	0.9993	0.9986	0.9989	0.9987	0.9980	1.0000
Logistic Regression	B	0.7535	0.9173	0.6231	0.7421	0.6658	0.5581	0.7799
Decision Tree	B	0.8117	0.9665	0.6932	0.8074	0.7348	0.6664	0.8309
Random Forest	B	0.7765	0.9687	0.6277	0.7618	0.6752	0.6167	0.9583

Table 1. Binary detectors under Protocol A (random split within the training distribution) and Protocol B (official held-out KDDTest+). Leakage-free pipelines; fixed seeds.

Protocol A reproduces the tutorial-level picture — every model close to perfect. Under Protocol B the same models lose 18–22 accuracy points; recall collapses to 0.62–0.69 while precision stays above 0.91, i.e., the models stay quiet and miss more than a third of the attacks. Bootstrap 95% recall intervals are narrow here (e.g. random forest 0.619–0.636), so this gap is not sampling noise. The next table shows the same phenomenon inside the article's own per-class design, and separates "was the test set used?" from "was the novelty gap tested?".

Class	Article (Table IV): acc / prec / rec (%)	CV on train (%)	CV on test (%)	Held-out KDDTest+: acc / recall (95% CI)
DoS	99.90 / 99.69 / 99.79	99.6	99.6	88.1 / 73.7 (72.7–74.6)

Probe	99.80 / 99.37 / 99.37	99.4	99.2	88.4 / 55.2 (53.2–57.1)
R2L	99.88 / 97.40 / 97.41	99.8	97.7	79.6 / 11.1 (9.9–12.3)
U2R	99.95 / 99.70 / 99.69	99.9	99.7	99.5 / 34.3 (22.4–44.8)

Table 2. The article's Table IV versus our faithful reproduction (ANOVA + RFE + decision tree, the article's per-class feature counts). CV on the test set reproduces the article (R2L 97.7% vs. a claimed 97.4%); held out honestly, recall collapses. The U2R interval is wide because KDDTest+ has only 67 U2R records.

Error analysis. For the random forest under Protocol B: per-category recall is 82.7% (DoS), 72.9% (Probe), 4.0% (R2L) and 11.9% (U2R); recall on attack types seen in training is 76.5% versus 29.6% on novel types (95% CI 28.1–31.2%). The most-missed attacks are *guess_passwd* (all 1,231 instances missed), *warezmaster* (837), *processtable* (678), *mscan* (653) and *snmpguess* (331). The pattern is systematic, not random: misses concentrate (a) where training data was scarce — R2L and U2R had 995 and 52 training records — and (b) on novel types, including novel *DoS* attacks (*apache2*, *mailbomb*, *processtable*) despite near-perfect detection of known DoS: the models learned the signatures of *neptune* and *smurf*, not the concept of a flood. The cybersecurity implication is uncomfortable: the detector is weakest exactly where an adversary would operate — credential attacks that look like legitimate logins (R2L sessions terminate with a normal *SF* flag 97% of the time) and techniques invented after training-data collection. **False-positive/false-negative trade-off:** at the default threshold the forest raises 260 false alarms per 9,711 benign connections (2.7%) while missing ≈4,800 of 12,833 attacks. Given the cost asymmetry (analyst minutes versus a breach), and a healthy ranking reserve (ROC-AUC 0.958 despite 0.63 recall), the operating point should be moved toward recall — the F2 metric and a threshold tuned against an explicit cost matrix formalize this. The decision tree has no such reserve (AUC 0.831): its limitation is the model, not the threshold.

6. Conclusions

Key findings. The article’s numbers are reproducible under its own protocol — and the test set is genuinely involved, because it is cross-validated rather than held out (CV on *KDDTest+* reproduces the article’s R2L figure, 97.7% vs. 97.4%). Held out as the benchmark intends, the same faithful pipeline detects only 11.1% of R2L and 34.3% of U2R attacks. The gap is driven, in order, by novel attack types (29.2% of test attacks), extreme class imbalance (995 R2L / 52 U2R training records), and feature selection tuned to known-attack signatures. Claim verdicts: C1 confirmed in-distribution (11–15 of 121 columns preserve performance) but untested under shift; C2 reproducible yet not generalizable (held-out R2L recall ≈11% vs. a claimed 97.4%); C3 confirmed; Q4 — our deployability question, not the article’s claim — not supported for R2L, U2R and novel attacks.

Lessons learned. The evaluation protocol matters more than the model: three different classifiers all looked near-perfect within one distribution and all dropped sharply on the held-out split. Cross-validation answers “did I fit this distribution?”, not “will this work on next year’s attacks?”. Accuracy under imbalance is close to meaningless — a U2R detector with 99.5% accuracy misses two of three attacks — and on rare classes results must carry confidence intervals (U2R held-out recall is 34% with a 95% interval of roughly ±11 points). Reproduction work pays: every individual number checks out, while the headline conclusion does not generalize.

Strengths of the proposed solution. A real and clearly stated problem; transparent, standard tooling; a faithful, interpretable feature-selection outcome that independent runs largely re-derive;

an honest, confirmed speed result.

Weaknesses. The evaluation pools the official test set through cross-validation instead of holding it out, so the train→test novelty gap — the benchmark's whole purpose — is never exercised; the per-class binary framing flatters the task; imbalance is unaddressed; accuracy leads the reporting; and the dataset bookkeeping is internally inconsistent.

Suggestions for future improvement. Evaluate on *KDDTest+*/*KDDTest-21* as the primary benchmark; handle imbalance (class weights, resampling, cost-sensitive learning); tune thresholds against an explicit FP/FN cost matrix; add an anomaly-detection branch (e.g., Isolation Forest trained on normal traffic) for never-seen attack families; calibrate scores; and validate on modern traffic datasets.

7. Executive Summary

This project critically evaluates Nkiama, Said and Saidu (2016), “A Subset Feature Elimination Mechanism for Intrusion Detection System” (IJACSA 7(4)), together with its community reference implementation, on the NSL-KDD network-intrusion benchmark. The article proposes per-attack-class feature selection — an ANOVA univariate filter followed by Recursive Feature Elimination around a decision tree — and reports approximately 99.7–99.9% accuracy, precision and recall with only 11–15 of 121 encoded features, plus an order-of-magnitude training speed-up.

We rebuilt the full pipeline in a single reproducible notebook (fixed seeds, vendored data, pinned dependencies, leakage-free pipelines) and scored it under two protocols: the article’s own (10-fold cross-validation within a single distribution) and the benchmark’s intended one (train on *KDDTrain+*, evaluate on the official *KDDTest+*, in which 29.2% of attack records belong to 17 attack types absent from training). Under the article’s protocol we reproduce its results essentially exactly — for the U2R class to the second decimal (99.9%). Under the intended held-out protocol the same faithful detectors collapse: recall falls to 73.7% (DoS), 55.2% (Probe), 11.1% (R2L, against a claimed 97.4%) and 34.3% (U2R), the last two with wide bootstrap intervals because those classes have 2,885 and only 67 test records. A three-model binary ensemble shows the same pattern (random forest: 99.9% random-split accuracy versus 62.8% held-out recall). The misses are systematic: recall on novel attack types is 29.6% versus 76.5% on known types, and the single most-missed attack is password guessing — all 1,231 instances — because R2L traffic is statistically almost indistinguishable from legitimate sessions in NSL-KDD’s features.

Diagnosis: the article uses the test set, but cross-validates it rather than holding it out. We demonstrate the mechanism directly — running cross-validation on *KDDTest+* itself (the procedure in the third-party reference implementation) reproduces the article’s R2L figure to within 0.3 points — while being careful not to attribute that exact procedure to the authors, who released no code. Either way the consequence holds: the published numbers are reproducible but do not test generalization to held-out attacks. The speed claim (C3) is confirmed; the in-distribution performance claim (C2) reproduces but does not generalize; and we would not deploy this pipeline as published without honest held-out evaluation, imbalance handling, cost-aware thresholds and an anomaly-detection branch for unseen attacks — though its interpretable feature-selection core remains a useful component. The broader lesson: in security machine learning, the evaluation protocol — not the classifier — is where the truth lives.

8. Summing It Up

The problem: deciding which of NSL-KDD’s 41 traffic features an intrusion-detection classifier actually needs. The source: Nkiama et al. (2016), who answer with a per-class ANOVA + RFE + decision-tree pipeline and report ≈99.9% across all metrics; we also audited its reference implementation on GitHub. The dataset: NSL-KDD, whose official test set was deliberately constructed to contain attack types unseen in training. The methodology of our reproduction study: rebuild the pipeline end-to-end, reproduce the article’s numbers under its own cross-validation protocol, then re-score the identical models on the official test set, with three binary baselines (logistic regression, decision tree, random forest) as controls and a systematic error analysis.

Main findings: the published numbers reproduce almost exactly under the published protocol — and the protocol is the problem. The article evaluates each attack class as a binary problem under a 10-fold cross-validation that pools the data; cross-validating *KDDTest+* itself reproduces its

figures (R2L 97.7% vs. 97.4%), so the published metrics do not exercise the train→test novelty gap. Held out as intended, R2L recall is 11.1% versus the claimed 97.4%, U2R recall is 34.3% (with a wide interval, only 67 test records), and every model's recall on novel attack types is less than half its recall on known ones. The author's performance claims are therefore **not supported under held-out evaluation**, with one confirmed exception: the training speed-up. The most important insights: evaluation protocol dominates model choice; accuracy under imbalance misleads (99.5% accuracy alongside 34% recall); rare-class results need confidence intervals; and feature selection, while interpretable and fast, encoded signatures of known attacks rather than transferable structure. Recommendation: do not use this pipeline as published for similar problems; reuse its feature-selection core only inside an honest held-out protocol with imbalance handling, cost-aware thresholds and an anomaly-detection component. Final conclusion: the article is a textbook case — reproducible numbers measured under a protocol that does not test generalization — and NSL-KDD's held-out test set, used as intended, is the instrument that reveals the difference.